Worksheet: Refactoring a Hello World application using Dependency Injection

based on a presentation by Sang Shin ex-Sun Java Evangelist

This worksheet explores the ideas of dependency injection and the Spring framework by exploring step-by-step refactoring processes for a standard `Hello World` application.

Lab Exercises

1. Import all the sample code for this lab
2. Build and Run `HelloWorld` sample application
3. Build and Run `HelloWorldWithCommandLineArguments` sample application
4. Build and Run `HelloWorldDecoupled` sample application
5. Build and Run `HelloWorldDecoupledInterface` sample application
6. Build and Run `HelloWorldDecoupledInterfaceWithFactory` sample application
7. Build and Run `HelloWorldSpring` sample application
8. Build and Run `HelloWorldSpringWithDI` sample application
9. Build and Run `HelloWorldSpringWithDIXMLFile` sample application
10. Build and Run `HelloWorldSpringWithDIXMLFileConstructorArgument`
11. Build and Run `HelloWorldSpringWithAnnotation` sample application
12. Build and Run `HelloWorldSpringWithAutoscan` sample application

Some of the images are taken from an Eclipse project but the equivalent can be easily achieved in IntelliJ or Netbeans.

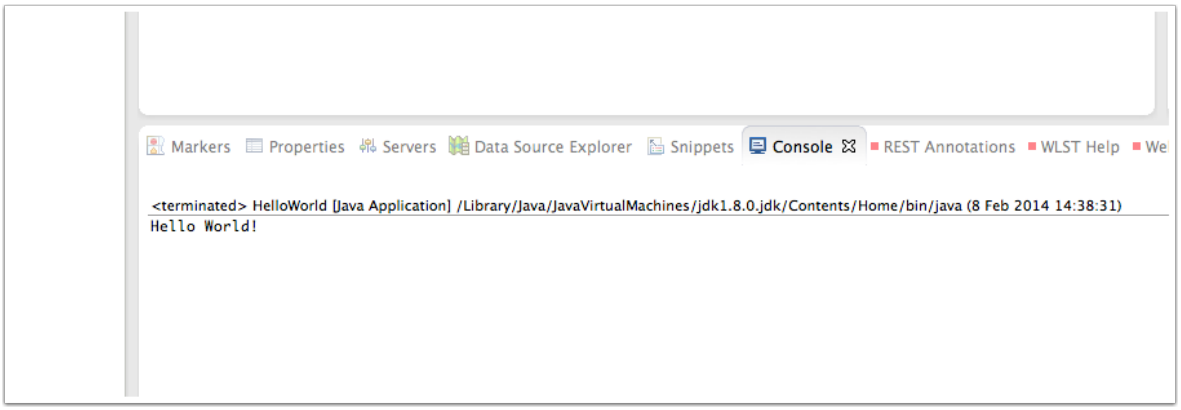
1 Download the source code repository

1. Download the repository which includes the examples for this worksheet. The relevant code will be under the folders `di` and `springdi`.
2. Create a new project called `helloworld` (or whatever name of your choice). As there are several versions of the code it is worth spending time organising the code into sub-projects (modules).

2 Build and Run `HelloWorld` sample application

In this exercise, you are going to build the good old `HelloWorld` application you probably wrote as your first Java application many years ago...

1. As most IDEs perform *incremental compilation* there is no need for a separate compilation step.
2. Run the application.
3. Observe the result.



4. Open and study `HelloWorld.java`.
   - Expand `HelloWorld->src` (or whatever the project is called).
   - Expand `helloworld`
   - Double-click `HelloWorld.java`.

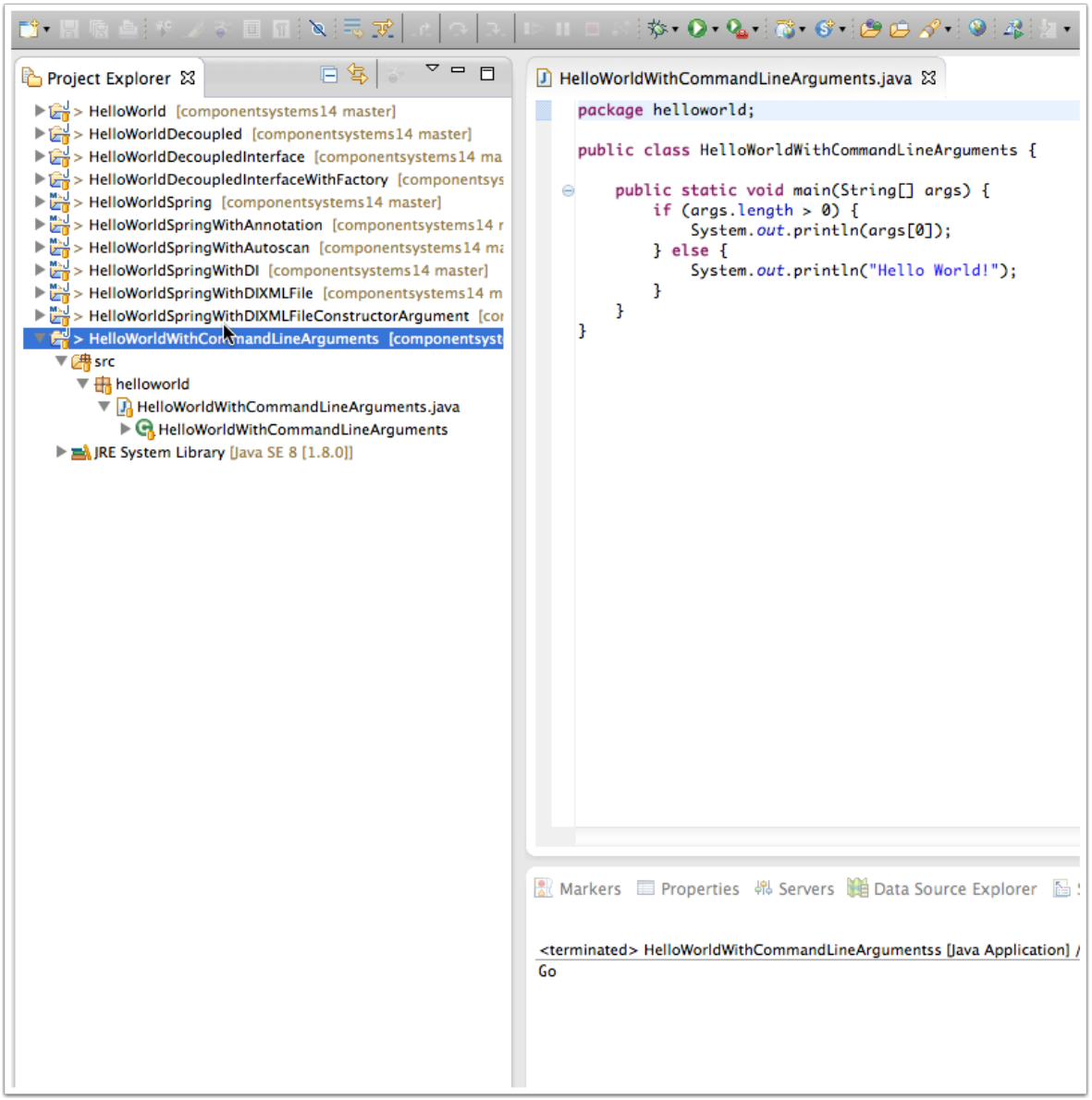
HelloWorld.java

3 Build and Run `HelloWorldWithCommandLineArguments` sample application

This code externalises the message content and read it in at runtime, from the command line argument. Now you can change the message without changing and recompiling the code.

1. Study `HelloWorldWithCommandLineArguments.java`
2. Run the application.

3. Observe that the text is displayed in the Console.



4 Build and Run `HelloWorldDecoupled` sample application

For this code, the message provider logic and the message renderer logic are separated from the rest of the code. So message provider (`HelloWorldMessageProvider`) can change without affecting the rest of the application (`StandardOutMessageRenderer`, `Launcher`) and the message renderer (`StandardOutMessageRenderer`) can change without affecting the rest of the application (`HelloWorldMessageProvider`, `Launcher`) .

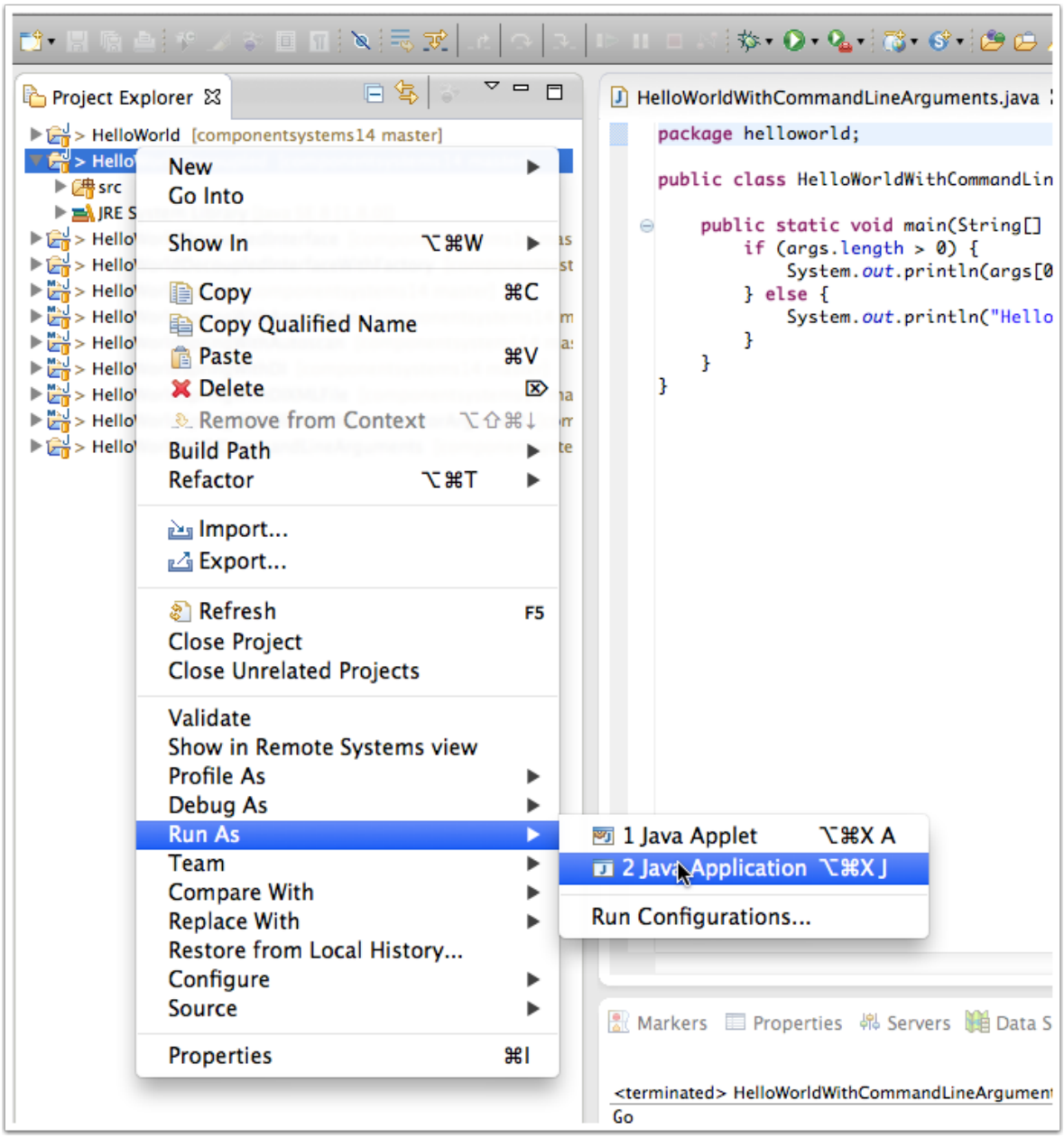
1. Study the `HelloWorldDecoupled` sample application

HelloWorldDecoupled.java

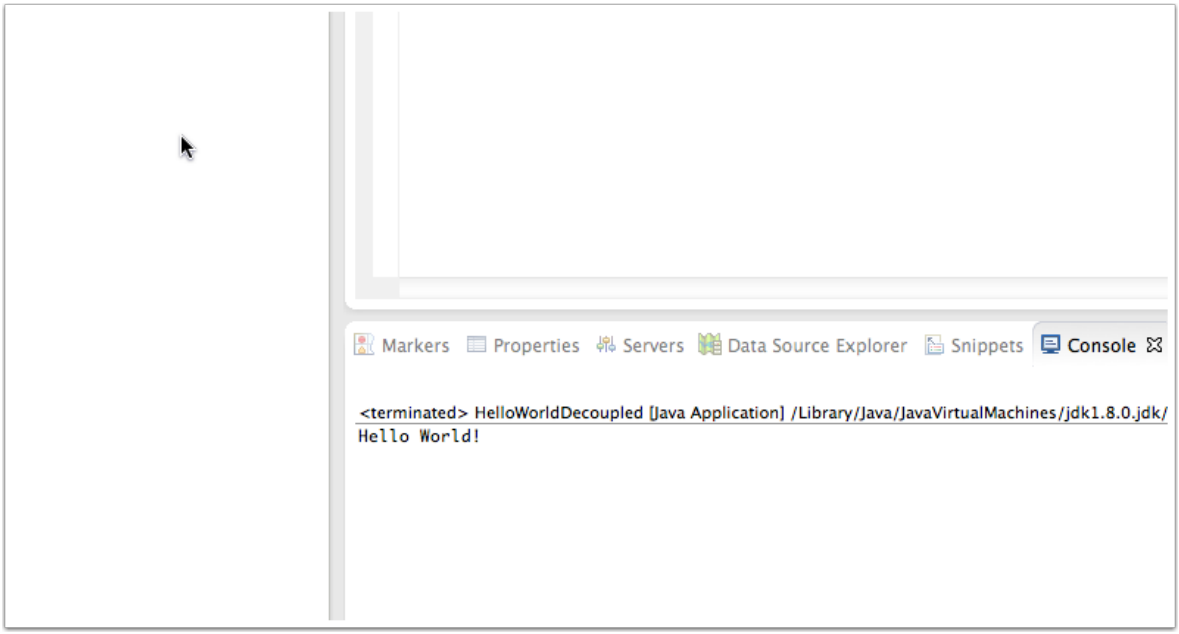
StandardOutMessageRenderer.java

HelloWorldMessageProvider.java

2. Run `HelloWorldDecoupled` sample application



3. Observe that the text is displayed in the Console.



5 Build and Run `HelloWorldDecoupledInterface` sample application

Now we are going to introduce Java interfaces so that the message rendering logic does not get affected by the change in message provider implementation.

1. Study the `HelloWorldDecoupledInterface` sample application

HelloWorldDecoupledInterface.java

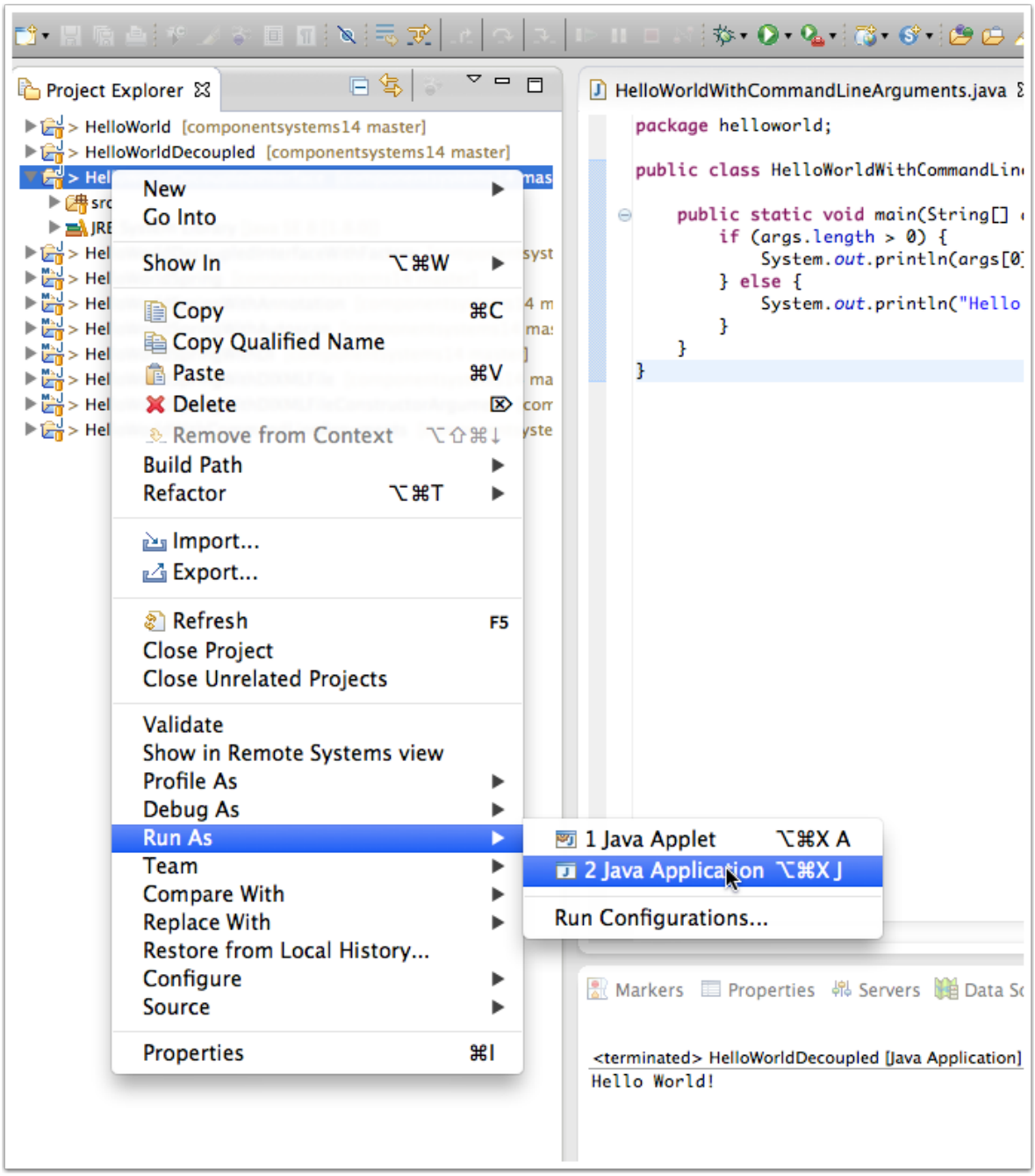
StandardOutMessageRenderer.java

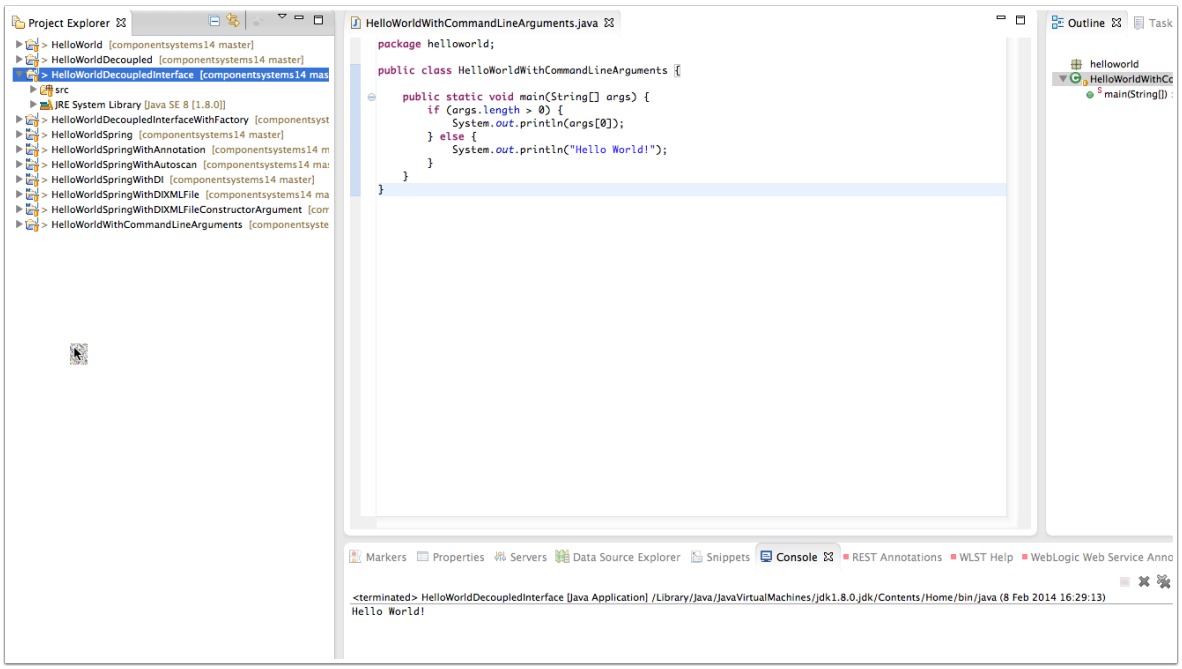
MessageRenderer.java

HelloWorldMessageProvider.java

MessageProvider.java

2. Run `HelloWorldDecoupledInterface` sample application





6 Build and Run `HelloWorldDecoupledInterfaceWithFactory` sample application

In this step, the properties file is used so that the message provider implementation and the message renderer implementation can be replaced simply by changing the properties file. So no change is required in the business logic code (launcher).

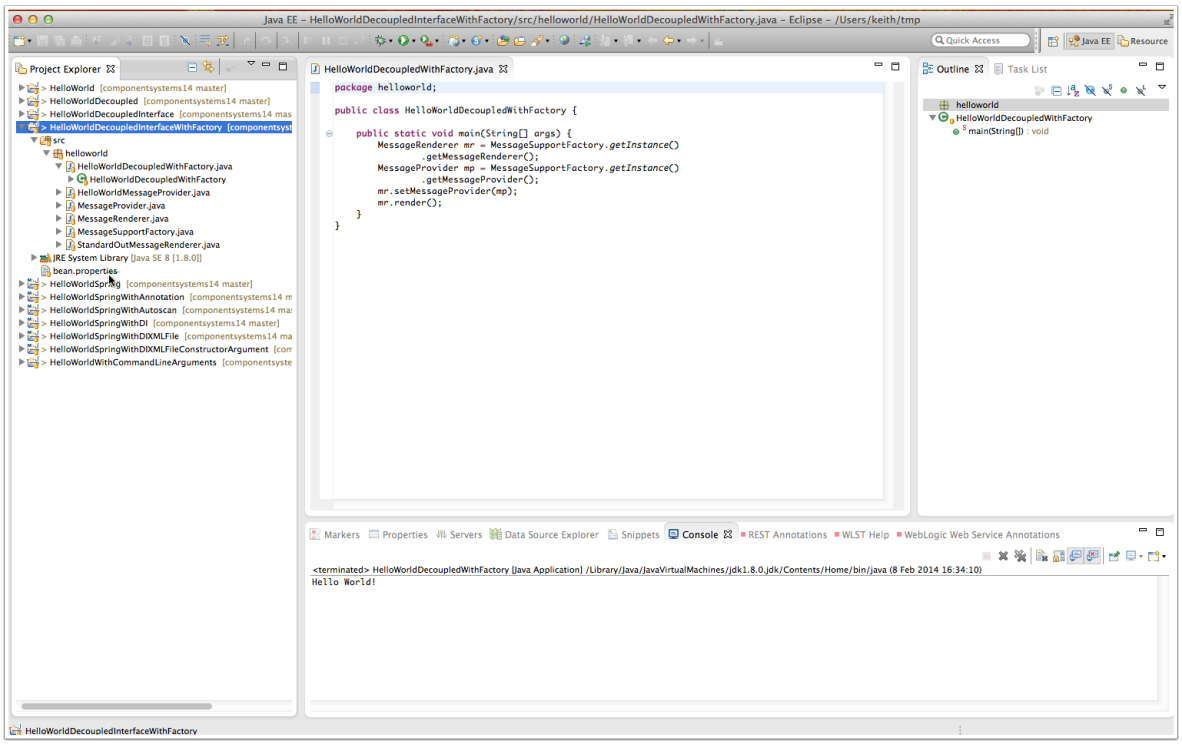
1. Study the `HelloWorldDecoupledInterface` sample application

HelloWorldDecoupledWithFactory.java

MessageSupportFactory.java

bean.properties

2. Run `HelloWorldDecoupledInterfaceWithFactory` sample application



7 Build and Run `HelloWorldSpring` sample application

Now you are going to remove the need of your own *glue code* (`MessageSupportFactory`) and use the Spring provided factory class instead.

1. Study the `HelloWorldSpring` sample application

HelloWorldSpring.java

beans.properties

2. Run `HelloWorldSpring` sample application

8 Build and Run `HelloWorldSpringWithDI` sample application

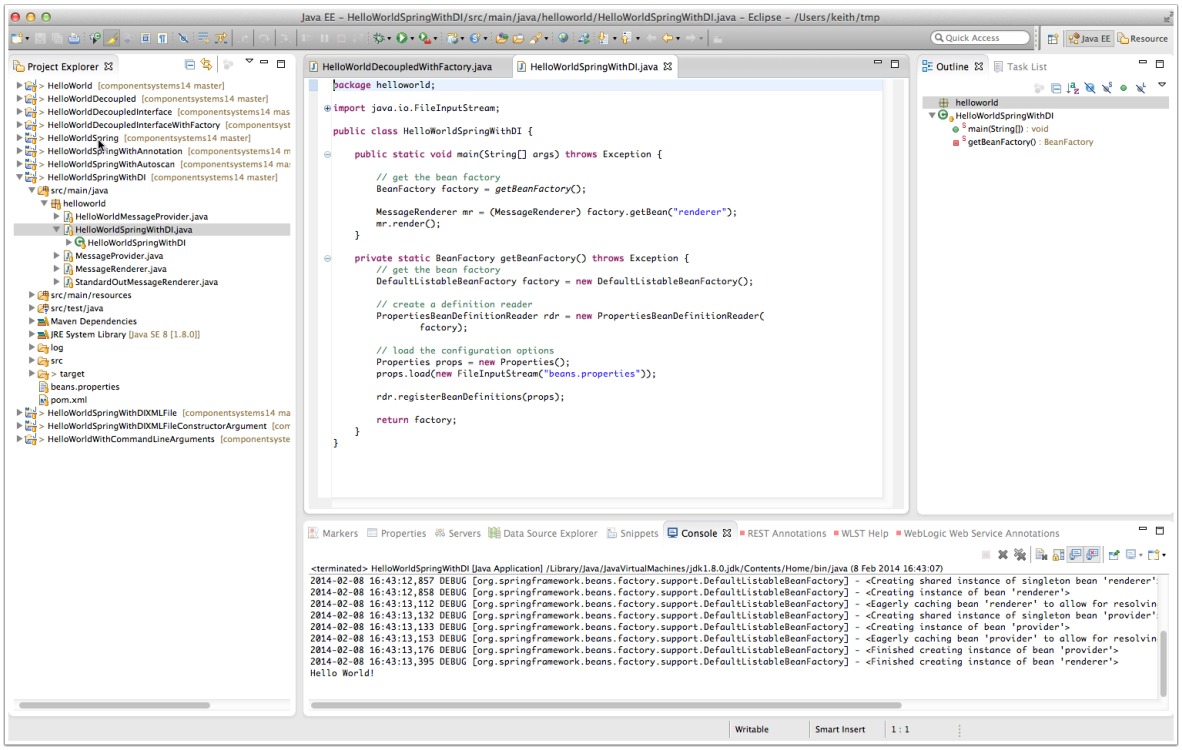
In this step you are going to use the Spring framework Dependency Injection (DI) mechanism. The `main()` method now just obtains the `MessageRenderer` bean and calls `render()`. It does not have to obtain the `MessageProvider` bean and set the `MessageProvider` property of the `MessageRenderer` bean itself because the *wiring* is performed through the Spring framework's Dependency Injection mechanism.

1. Study the `HelloWorldSpringWithDI` sample application

HelloWorldSpringWithDI.java

beans.properties

2. Run `HelloWorldSpringWithDI` sample application



9 Build and Run `HelloWorldSpringWithDIXMLFile` sample application

In this step, you are going to use an XML file in which the wiring requirements are specified.

1. Study the `HelloWorldSpringWithDIXMLFile` sample application

HelloWorldSpringWithDIXMLFile.java

2. Run `HelloWorldSpringWithDIXMLFile` sample application

10 Build and Run `HelloWorldSpringWithDIXMLFileConstructorArgument` sample application

In this step, you are going to change the wiring requirement using the constructor.

1. Study the `HelloWorldSpringWithDIXMLFileConstructorArgument` sample application

HelloWorldSpringWithDIXMLFileConstructorArgument.java

beans.xml

ConfigurableMessageProvider.java

2. Run `HelloWorldSpringWithDIXMLFileConstructorArgument` sample application

11 Build and Run `HelloWorldSpringWithAnnotation` sample application

In this step, you are going to use Java annotations as the injection mechanism.

1. Study the code

StandardOutMessageRenderer.java

beans.xml

2. Run `HelloWorldSpringWithAnnotation` sample application

12 Build and Run `HelloWorldSpringWithAutoscan` sample application

In this step, you are going to use the *autoscan* feature of Spring DI.

1. Study the code

beans.xml

StandardOutMessageRenderer.java

HelloWorldMessageProvider.java

2. Run `HelloWorldSpringWithAutoscan` sample application